

Week 1- Fundamentals

Condensed Study Notes

Generated: 2026-05-25 00:21 UTC

Week 1- Fundamentals

Welcome to Week 1, focusing on the fundamentals.

Instructors for this week are Dr. F Adamu-Fika and Ms. OV Okpoko.

Learning Objectives

This section outlines what you will learn:

- **Define Algorithms:** Understand what algorithms are and why they are crucial in computing.
- **Classify by Paradigm:** Explore different algorithmic approaches, such as divide-and-conquer and greedy methods.
- **Classify by Function:** Learn to categorize algorithms based on the problems they solve, like sorting or searching.
- **Foundation for Analysis:** Prepare to analyze algorithms for their efficiency and complexity.

Definition of an Algorithm

An algorithm is a step-by-step procedure or formula for solving a problem.

Key Characteristics:

- **Input:** Takes zero or more inputs.
- **Output:** Produces at least one output.
- **Definiteness:** Steps are clear and unambiguous.
- **Effectiveness:** All steps are basic enough to be carried out.
- **Finiteness:** Terminates after a finite number of steps.

Examples include sorting, searching, and finding the shortest path.

Importance and application of algorithms

Algorithms are fundamental to computer science and software development, enabling efficient problem-solving across diverse domains.

Key Applications:

- **Navigation:** Finding the best routes (e.g., GPS).
- **E-commerce:** Powering search functions and product recommendations.
- **Cybersecurity:** Implementing encryption and hashing techniques.

- **General Domains:** Applicable to networks, Artificial Intelligence (AI), and databases.

Classification by Function or Problem Type

Algorithms can be classified by the specific type of problem they solve. This classification helps in choosing the right tool for a given task.

Key types include:

- **Searching Algorithms:** Locate a specific piece of data within a larger set. Examples: Binary Search, Linear Search.
- **Sorting Algorithms:** Arrange data into a defined order. Examples: Quick Sort, Merge Sort.
- **Graph Algorithms:** Address problems involving data structured as graphs (nodes connected by edges). Examples: Breadth-First Search (BFS), Depth-First Search (DFS), Dijkstra's algorithm.
- **Optimisation Algorithms:** Find the best possible solution when faced with certain limitations or constraints. Example: Knapsack problem.
- **String Algorithms:** Process and manipulate text or sequences of characters. Example: Knuth-Morris-Pratt (KMP) for pattern matching.
- **Decision Problems:** Answer a specific question with a simple 'yes' or 'no'. Example: Satisfiability problem.
- **Combinatorial Algorithms:** Solve problems involving combinations, permutations, or subsets of data. Example: Traveling Salesman Problem.
- **Numerical Algorithms:** Perform mathematical calculations, such as finding roots of equations or approximating integrals. Example: Newton-Raphson method.

Classification by Algorithmic Paradigm

Algorithmic paradigms are high-level strategies for solving computational problems.

- **Brute Force:** Exhaustively searches all possible solutions.
 - Example: Checking all subsets for the subset-sum problem.
- **Divide-and-Conquer:** Breaks a problem into smaller, independent subproblems, solves them, and combines their results.
 - Examples: Merge Sort, Quick Sort.
- **Decrease-and-Conquer:** Reduces a problem to a smaller instance and builds up a solution.
 - Types:
 - **Decrease by a Constant:** Example: Insertion Sort.
 - **Decrease by a Factor:** Example: Binary Search.
 - **Decrease by Variable Size:** Example: Euclidean algorithm for GCD.
- **Dynamic Programming:** Solves problems by breaking them into overlapping subproblems and storing results to avoid redundant calculations.
 - Examples: Fibonacci sequence, Knapsack problem.

- **Greedy Approach:** Makes the locally optimal choice at each step, aiming for an overall optimal solution.
- Examples: Dijkstra's Algorithm (shortest paths with non-negative weights), Prim's Algorithm (MST), Huffman Coding (data compression).
- **Transform-and-Conquer:** Transforms the problem into another form that is easier to solve.
- Types:
 - **Representation Change:** Example: Balanced search trees.
 - **Problem Reduction:** Example: Transforming shortest path problems.
 - **Instance Simplification:** Example: Heapsort.
- **Backtracking:** Incrementally builds solutions and abandons a candidate as soon as it determines it cannot be valid.
- Examples: N-Queens problem, Sudoku solver.
- **Approximation Algorithms:** Find approximate solutions when exact solutions are computationally infeasible (e.g., for NP-hard problems).
- Example: Approximation for Traveling Salesman Problem.

Structure of Algorithms

The structure of an algorithm refers to how it is organized, rather than what it does.

- **Iterative Structure:** This structure uses loops to repeat a set of steps until a specific condition is met.
- **Recursive Structure:** An algorithm employing this structure calls itself to solve smaller versions of the same problem.

Note: The provided content mentions 'Recursive factorial calculation' and 'Iterative factorial calculation' as examples, but does not detail their specific structures within this section.

Examples of Recursive structure

Recursive structures in algorithms involve a function calling itself to solve smaller instances of the same problem.

Consider the factorial function, denoted as $n!$. It's defined as $n! = n * (n-1)!$ for $n > 0$, and $0! = 1$.

This definition is inherently recursive:

- To calculate $5!$, we need $5 * 4!$.
- To calculate $4!$, we need $4 * 3!$.
- This continues until we reach the base case, $0! = 1$.

Once the base case is hit, the results are combined back up the chain of calls.

Another example is the Fibonacci sequence, where $F(n) = F(n-1) + F(n-2)$ for $n > 1$, with base cases $F(0) = 0$ and $F(1) = 1$.

Each recursive call breaks the problem down into smaller, identical sub-problems until a simple, solvable base case is reached. The solutions to these sub-problems are then combined to solve the original problem.

Algorithm EVENSUM

Algorithm EVENSUM(n) processes a positive integer ' n ' to calculate a sum.

Input:

- n : a positive integer.

Output:

- The sum of the n th even natural number. The sequence of even natural numbers starts with 0 (i.e., 0, 2, 4, 6, ...).

START

The algorithm continues with a loop condition: 3. while ($n \leq n$).

Inside the loop, the variable 'even' is updated: 4. $even = even + (2n - 2)$.

The algorithm concludes by returning a value: 4. return EVENSUM $n - 1 + (2n - 2)$ END.

Examples of Iterative structure

This section provides examples to illustrate the iterative structure of algorithms. Iterative structures involve repeating a set of operations until a specific condition is met, typically using loops.

Previously, we saw the EVENSUM(n) algorithm detailed its iterative completion. This demonstrated how a loop continues, variables are updated, and a final result is returned, all characteristic of an iterative approach.

Pros and Cons

Recursive algorithms can be easier to read but often consume more memory.

Iterative algorithms are typically more efficient in their use of memory space.

Efficiency and Complexity

Efficiency refers to how quickly and effectively an algorithm solves a problem. Complexity is understood to help conserve resources, particularly with large datasets.

Two primary measures of complexity are:

- **Time Complexity:** Measures the runtime of an algorithm, viewed as a function of the input size.
- **Space Complexity:** Measures the memory usage of an algorithm.

Real-World Examples of Algorithms

Algorithms are fundamental to many real-world applications:

- **Pathfinding:** Dijkstra's Algorithm is used in GPS navigation systems to find the shortest route.
- **Sorting:** Algorithms like Quick Sort are employed for efficient database indexing, organizing data for quick retrieval.
- **Searching:** Binary Search is a core component of search engines, enabling rapid location of information within large datasets.
- **Cybersecurity:** Encryption algorithms are vital for securing data transmission, ensuring privacy and integrity.

Online resources

Here are some recommended online resources to supplement your learning:

- **VisuAlgo:** Offers visual representations and interactive animations of algorithms, excellent for understanding recursive and iterative structures.
- **MIT OpenCourseWare – “Introduction to Algorithms”:** A free course based on the CLRS textbook, covering algorithms, paradigms, and complexity.
- **GeeksforGeeks – Basics of Algorithms:** Provides articles, sample code, and explanations on introductory algorithms like sorting, searching, and recursion/iteration.
- **Coursera – “Algorithms Specialization” by Stanford University:** Week 1 focuses on fundamental concepts and algorithmic thinking.
- **Khan Academy – “Algorithms”:** Beginner-friendly tutorials on basic concepts, classifications, and introductory complexity analysis, with interactive exercises.